



IS231: Web Technology JavaScript and AJAX



By: Neamat El Tazi

References

- **W3schools.com**
- <https://www.geeksforgeeks.org/mvc-design-pattern/>
- <https://selftaughtcoders.com/from-idea-to-launch/lesson-17/laravel-5-mvc-application-in-10-minutes/>

ECMAScript 2009 – ES5

- ▶ ECMAScript 2009, also known as ES5, was the first major revision to JavaScript.

ES5 Most Important Features

- ▶ "use strict"
- ▶ String.trim()
- ▶ Array.isArray()
- ▶ Array.forEach()
- ▶ Array.map()
- ▶ Array.filter()
- ▶ Array.reduce()
- ▶ Array.reduceRight()
- ▶ Array.every()
- ▶ Array.some()
- ▶ Array.indexOf()
- ▶ Array.lastIndexOf()
- ▶ JSON.parse()
- ▶ JSON.stringify()
- ▶ Date.now()
- ▶ Property Getters and Setters
- ▶ New Object Property Methods

Use Strict Directive

- ▶ The "use strict" directive was new in ES5.
- ▶ It is not a statement, but a literal expression, ignored by earlier versions of JavaScript.
- ▶ The purpose of "use strict" is to indicate that the code should be executed in "strict mode".
- ▶ With strict mode, you can not, for example, use undeclared variables.

```
"use strict";  
x = 3.14;      // This will cause an error because x is not declared
```

String.trim()

- ▶ String.trim() removes whitespace from both sides of a string.

```
var str = "      Hello World!      ";  
alert(str.trim());
```

- ▶ Check Demo 11

Array.isArray()

- ▶ The `isArray()` method checks whether an object is an array.

```
var fruits =  
["Banana", "Orange", "Apple", "Mango"];  
console.log(Array.isArray(fruits));
```

Check Demo 12

Array.forEach()

- ▶ The `forEach()` method calls a function once for each array element.

```
function myFunction(value) {  
    txt = txt + value + "<br>";  
    console.log(txt);  
}
```

```
var txt = "";  
var numbers = [45, 4, 9, 16, 25];  
numbers.forEach(myFunction);
```

Check Demo 13

Array.map()

- ▶ The map() method creates a new array by performing a function on each array element.
- ▶ The map() method does not execute the function for array elements without values.
- ▶ The map() method does not change the original array.
- ▶ This example multiplies each array value by 2:

```
function myFunction(value) {  
  return value * 2;  
}
```

```
var numbers1 = [45, 4, 9, 16, 25];  
var numbers2 = numbers1.map(myFunction);
```

Check Demo 14

Array.filter()

- ▶ The `filter()` method creates a new array with array elements that passes a test.
- ▶ This example creates a new array from elements with a value larger than 18:

```
var numbers = [45, 4, 9, 16, 25];  
var over18 = numbers.filter(myFunction);
```

```
function myFunction(value) {  
    return value > 18;  
}
```

###check Arrow functions in ES6 .. Coming 😊

Array.reduce()

- ▶ The reduce() method runs a function on each array element to produce (reduce it to) a single value.
- ▶ The reduce() method works from left-to-right in the array
- ▶ This example finds the sum of all numbers in an array:

```
var numbers1 = [45, 4, 9, 16, 25];  
var sum = numbers1.reduce(myFunction);  
  
function myFunction(total, value) {  
    return total + value;  
}
```

//Total here represents the initial value or the previously returned value
Check Demo 16

Array.reduceRight()

- ▶ The `reduceRight()` method runs a function on each array element to produce (reduce it to) a single value.
- ▶ The `reduceRight()` works from right-to-left in the array.
- ▶ This example finds the sum of all numbers in an array:

```
var numbers1 = [45, 4, 9, 16, 25];  
var sum = numbers1.reduceRight(myFunction);  
  
function myFunction(total, value, index, array)  
{  
    return total + value;  
}
```

Array.every()

- ▶ The every() method checks if all array values pass a test.
- ▶ This example check if all array values are larger than 18:

```
var numbers = [45, 4, 9, 16, 25];  
var allOver18 = numbers.every(myFunction);  
  
function myFunction(value, index, array) {  
    return value > 18;  
}
```

Check Demo 18

Array.some()

- ▶ The some() method check if some array values pass a test.
- ▶ This example check if some array values are larger than 18:

```
var numbers = [45, 4, 9, 16, 25];  
var someOver18 = numbers.some(myFunction);  
  
function myFunction(value, index, array) {  
    return value > 18;  
}
```

Check Demo 18 and change the function to “some”

Array.indexOf()

- ▶ The `indexOf()` method searches an array for an element value and returns its position.
- ▶ Note: The first item has position 0, the second item has position 1, and so on.

```
var fruits =  
["Apple", "Orange", "Apple", "Mango"];  
var a = fruits.indexOf("Apple");
```

Array.lastIndexOf()

- ▶ `Array.lastIndexOf()` is the same as `Array.indexOf()`, but returns the position of the last occurrence of the specified element.

```
var fruits =  
["Apple", "Orange", "Apple", "Mango"];  
var a = fruits.lastIndexOf("Apple");
```


Array.find()

- ▶ The find() method returns the value of the first array element that passes a test function.
- ▶ This example finds (returns the value of) the first element that is larger than 18:

```
var numbers = [4, 9, 16, 25, 29];  
var first = numbers.find(myFunction);  
  
function myFunction(value, index, array) {  
    return value > 18;  
}
```

Check Demo 18 and change the function to “find”

JSON.stringify() and JSON.parse()

- ▶ A common use of JSON is to send data to a web server.
- ▶ When sending data to a web server, the data has to be a string.
- ▶ Imagine we have this object in JavaScript:
 - ▶ `var obj = {name:"John", age:30, city:"New York"};`
- ▶ Use the JavaScript function `JSON.stringify()` to convert it into a string.
 - ▶ `var myJSON = JSON.stringify(obj);`

Date.now()

- ▶ `Date.now()` returns the number of milliseconds since zero date (January 1, 1970 00:00:00 UTC).
- ▶ `var timInMSs = Date.now();`
- ▶ `Date()` returns today's date and time now.

Property Getters and Setters

- ▶ ES5 lets you define object methods with a syntax that looks like getting or setting a property.
- ▶ This example creates a **getter** for a property called fullName:

```
// Create an object:
var person = {
  firstName: "John",
  lastName : "Doe",
  get fullName() {
    return this.firstName + " " + this.lastName;
  }
};

// Display data from the object using a getter:
document.getElementById("demo").innerHTML = person.fullName;
```

New Object Property Methods

- ▶ `Object.defineProperty()` is a new Object method in ES5.
- ▶ `// Create an Object:`

```
var person = {  
  firstName: "John",  
  lastName : "Doe",  
  language : "NO",  
};
```

`// Change a Property:`

```
Object.defineProperty(person, "language", {  
  value: "EN",  
  writable : true,  
  enumerable : true,  
  configurable : true  
});
```

`// Enumerate Properties`

```
var txt = "";  
for (var x in person) {  
  txt += person[x] + "<br>";  
}  
document.getElementById("demo").innerHTML = txt;
```

Property Access on Strings

- ▶ The `charAt()` method returns the character at a specified index (position) in a string:

```
var str = "HELLO WORLD";  
str.charAt(0);           // returns H
```

ES5 allows property access on strings:

```
var str = "HELLO WORLD";  
str[0];                  // returns H
```

Trailing Commas

- ▶ ES5 allows trailing commas in object and array definitions:



```
person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 46,  
}
```

- ▶ Note: JSON does not allow trailing commas.

// Not allowed:

```
var person = '{"firstName":"John",  
"lastName":"Doe", "age":46,}'  
JSON.parse(person)
```

Strings Over Multiple Lines

- ▶ ES5 allows string literals over multiple lines if escaped with a backslash:

```
"Hello \  
Dolly!";
```

A safer way to break up a string literal, is to use string addition:

```
"Hello " +  
"Dolly!";
```


Reserved Words as Property Names

- ▶ ES5 allows reserved words as property names:

```
var obj = {name: "John", new: "yes"}
```

ECMAScript 2015 – ES6

- ▶ ECMAScript 6, also known as ES6 and ECMAScript 2015, was the second major revision to JavaScript.

New Features in ES6

- ▶ The let keyword
- ▶ The const keyword
- ▶ JavaScript Arrow Functions
- ▶ JavaScript Class
- ▶ JavaScript Promise
- ▶ Default Parameter Values
- ▶ New Types Properties (check it on www.w3schools.com)
- ▶ ES7,8,9,10,11,12,13,14,15
- ▶ ES2024 = ES15
- ▶ ES2025 (latest Finalized as of now)

The Var Keyword

- ▶ Before ES6, the var keyword was used to declare a variable in JavaScript. The var keyword has been around since the inception of JavaScript, and it's what you will see in any pre ES6 code.
- ▶ Variables declared using the var keyword are either globally or functionally scoped, they do not support block-level scope. This means that if a variable is defined in a loop or in an if statement it can be accessed outside the block and accidentally redefined leading to a buggy program.

The Var Keyword

```
//var
var colour = 'red'
console.log(colour);           //red

function getColour() {
  console.log(colour);         //colour is accessible here
}

getColour();                   //red

if (true) {
  var greeting = 'hello!';

  console.log(greeting); // 'hello!';
}

console.log(greeting); // 'hello!';
```

(ES6) Let Keyword

- ▶ The let keyword is very similar to the var keyword as they both allow you to reassign the value later. The main difference between the two is that let deals with a block scope and although it can be reassigned it cannot be redeclared.
- ▶ Generally, you should avoid using the var keyword and use Let instead.

Let Keyword

```
//let
let number = 10;
number = 100000;

console.log(number); // Prints 100000

let number = 10;
let number = 100000;

console.log(number); // SyntaxError: Identifier 'number' has already been declared

let number = 50;
{ let number = 100000;

console.log(number); } // Prints 100000
```

JavaScript let keyword

- ▶ The let keyword allows you to declare a variable with block scope.

```
var x = 10;  
// Here x is 10  
{  
  let x = 2;  
  // Here x is 2  
}  
// Here x is 10
```


JavaScript const

- ▶ The `const` keyword allows you to declare a constant (a JavaScript variable with a constant value).
- ▶ Constants are similar to `let` variables, except that the value cannot be changed.

```
var x = 10;  
// Here x is 10  
{  
  const x = 2;  
  // Here x is 2  
}  
// Here x is 10
```

Arrow Functions

- ▶ Arrow functions allows a short syntax for writing function expressions.
- ▶ You don't need the function keyword, the return keyword, and the curly brackets.

```
// ES5  
var x = function(x, y) {  
    return x * y;  
}
```

```
// ES6  
const x = (x, y) => x * y;
```

Arrow Functions

- ▶ Arrow functions do not have their own **this**. They are not well suited for defining object methods.
- ▶ They must be defined **before** they are used.
- ▶ Using **const** is safer than using **var**, because a function expression is always a constant value.
- ▶ You can only omit the **return** keyword and the curly brackets if the function is a single statement.

```
const x = (x, y) => { return x * y };  
=  
const x = (x, y) => x * y;
```

Remember

//ES5

```
var numbers = [45, 4, 9, 16, 25];  
var over18 = numbers.filter(myFunction);
```

```
function myFunction(value) {  
    return value > 18;  
}
```

//ES6

```
var numbers = [45, 4, 9, 16, 25];  
var over18 = numbers.filter((value)=>value>18);
```

Check Demo 20

JavaScript Classes

- ▶ JavaScript Classes are templates for JavaScript Objects.
- ▶ Use the keyword `class` to create a class.
- ▶ Always add a method named `constructor()`:

Syntax

```
▶ class ClassName {  
    constructor() { ... }  
}
```

JavaScript Classes

- ▶ Classes were introduced in ES6 to provide a cleaner way to follow object-oriented programming patterns.
- ▶ JavaScript still follows a prototype-based inheritance model. Classes in JavaScript are syntactic sugar over the prototype-based inheritance model which we use to implement OOP concepts.
- ▶ Thus, the introduction of classes in JS made it easier for developers to build software around OOP concepts. It also brought in similarities to different OOP-based programming languages such as C++ and Java.

Before Classes

- ▶ Before classes, we used constructor functions to do OOP in JavaScript.

```
function Pen(name, color, price)
{
  this.name = name;
  this.color = color;
  this.price = price;
}
const pen1 = new Pen("Marker", "Blue", "$3");
console.log(pen1);
```

Before Classes – Adding a function to a constructor

```
function Pen(name, color, price) {  
    this.name = name;  
    this.color = color;  
    this.price = price;  
}
```

```
const pen1 = new Pen("Marker", "Blue", "$3");
```

```
Pen.prototype.showPrice = function(){  
    console.log(`Price of ${this.name} is ${this.price}`);  
}
```

```
pen1.showPrice();
```


Using Class Keyword

```
class Pen {  
    constructor(name, color, price){  
        this.name = name;  
        this.color = color;  
        this.price = price;  
    }  
  
    showPrice(){  
        console.log(`Price of ${this.name} is ${this.price}`);  
    }  
}  
  
const pen1 = new Pen("Marker", "Blue", "$3");  
pen1.showPrice();
```

JavaScript Classes – Another example

Example

```
▶ class Car {  
    constructor(name, year) {  
        this.name = name;  
        this.year = year;  
    }  
}
```

▶ Using a Class

```
let myCar1 = new Car("Ford", 2014);  
let myCar2 = new Car("Audi", 2019);
```

Inheritance in Javascript Classes

```
class OfficeSupplies {
  constructor(name, color, price){
    this.name = name; this.color = color; this.price = price;
  }
  showPrice(){ console.log(`Price of ${this.name} is ${this.price}`);
}
}

class Pen extends OfficeSupplies {
  constructor(type){
    super();
    this.type = type;
  }
  showType(){
    console.log(`Type of ${this.name} is ${this.type}`); }}
}
```

JavaScript Promise Object

- ▶ A Promise is a JavaScript object that links "Producing Code" and "Consuming Code".
- ▶ "Producing Code" can take some time and "Consuming Code" must wait for the result.

Promise Syntax

```
let myPromise = new Promise(function(myResolve, myReject) {  
  // "Producing Code" (May take some time)  
  
  myResolve(); // when successful  
  myReject();  // when error  
});  
  
// "Consuming Code" (Must wait for a fulfilled Promise).  
myPromise.then(  
  function(value) { /* code if successful */ },  
  function(error) { /* code if some error */ }  
);
```

Example Using a Promise

```
let myPromise
= new Promise(function(myResolve, myReject)
{
    setTimeout(function() { myResolve("I love
You !!"); }, 3000);
});
```

```
myPromise.then(function(value) {
    document.getElementById("demo").innerHTML
= value;
});
```

► Check Demo 200.html

Default Parameter Values

- ▶ ES6 allows function parameters to have default values.

```
function myFunction(x, y = 10) {  
    // y is 10 if not passed or undefined  
    return x + y;  
}  
myFunction(5); // will return 15
```

Function Rest Parameter

- ▶ The rest parameter (...) allows a function to treat an indefinite number of arguments as an array:

```
function sum(...args) {  
  let sum = 0;  
  for (let arg of args) sum += arg;  
  return sum;  
}
```

```
let x = sum(4, 9, 16, 25, 29, 100, 66, 77);
```

Check Demo 22

Array.find()

- ▶ The find() method returns the value of the first array element that passes a test function.
- ▶ This example finds (returns the value of) the first element that is larger than 18:

```
var numbers = [4, 9, 16, 25, 29];  
var first = numbers.find(myFunction);  
  
function myFunction(value) {  
    return value > 18;  
}
```


New Global Methods: The isNaN() Method

- ▶ The global isNaN() method returns true if the argument is NaN. Otherwise it returns false

```
isNaN("Hello");           // returns true
```

```
//Try it
```

New Global Methods: The isFinite() Method

- ▶ The global isFinite() method returns false if the argument is Infinity or NaN.
- ▶ Otherwise it returns true:

```
isFinite(10/0);    // returns false
isFinite(10/1);    // returns true
isFinite("hello"); // returns false
```

ECMA2016 – ES7

- ▶ Introduced only two new features:
 - ▶ `Array.prototype.includes()`
 - ▶ Exponential Operator

ES7 – Includes() feature

- ▶ In ES6, we used to use `indexOf()` function to check if a value exists in an array like the following:

```
Let numbers = [1,2,3,4]
```

```
if(numbers.indexOf(2) !== -1) { //do something }
```

- ▶ In ES7, we can write:

```
Let numbers = [1,2,3,4]
```

```
if(numbers.includes(2)) { //do something }
```

Difference between indexOf and includes

- ▶ `Array.prototype.includes()` handles NaN better than `Array.prototype.indexOf()`. If the array contains NaN, `indexOf()` cannot handle it.
- ▶ `Array.prototype.includes()` returns the correct value when searching for NaN.

```
console.log(numbers.indexOf(NaN)); //Prints -1
```

```
console.log(numbers.includes(NaN)); //Prints true
```

ES7 – Exponentiation Operator

- ▶ ECMAScript 2016 introduced the exponentiation operator, `**`.
- ▶ It has the same purpose as `Math.pow()`. It returns the first argument raised to the power of the second argument.

```
let base = 3;  
let exponent = 4;  
let result = base**exponent;  
console.log(result); //81
```

ES8

- ▶ ES8 introduces two string handling functions for padding a string:
 - ▶ `String.padStart()`
 - ▶ `String.padEnd()`
- ▶ Async and Await

ES8 – padStart()

<script>

string_value.padStart(targetLength [, padString])

//pad the String with 0

let product_cost = '1699'.padStart(7,0)

console.log(product_cost)

console.log(product_cost.length)

//pad the String with blank spaces

let product_cost1 = '1699'.padStart(7)

console.log(product_cost1)

console.log(product_cost1.length)

</script>

The output of the above code will be as stated below –

0001699

7

1699

7

ES8 – padEnd()

<script>

string_value.padEnd(targetLength [, padString])

//pad the string with x

let product_cost = '1699'.padEnd(7, 'x')

console.log(product_cost)

console.log(product_cost.length)

//pad the string with spaces

let product_cost1 = '1699'.padEnd(7)

console.log(product_cost1)

console.log(product_cost1.length)

</script>

The output of the above code will be as mentioned below –

1699xxx

7

1699

7

ES8 – Async and Await

- ▶ Async/Await is a very important feature in ES8.
- ▶ The await keyword is used with promises. This keyword can be used to pause the execution of a function till a promise is settled.
- ▶ The await keyword returns value of the promise if the promise is resolved while it throws an error if the promise is rejected.
- ▶ The await function can only be used inside functions marked as async. A function that is declared using the async keyword always returns a promise.

ES8 – Async and Await --- in ES6

The below code shows invoking the asynchronous function using the traditional ES6 syntax - `promise.then()`

```
<script>
  function fnTimeConsumingWork(){
    return new Promise((resolve,reject)=>{
      setTimeout(() => {
        resolve('response is:2 seconds have passed')
      }, 2000);
    })
  }

  fnTimeConsumingWork().then(resp=>{
    console.log(resp)
  })
  console.log('end of script')
</script>
```

ES8 – Async and Await

The below code shows a cleaner way of invoking the asynchronous function using ES8 syntax - async/await

```
<script>
  function fnTimeConsumingWork(){
    return new Promise((resolve,reject)=>{
      setTimeout(() => {
        resolve('response is:2 seconds have passed')
      }, 2000);
    })
  }
  async function my_AsyncFunc(){
    console.log('inside my_AsyncFunc')
    const response = await fnTimeConsumingWork();// clean and readable
    console.log(response)
  }
  my_AsyncFunc();
  console.log("end of script")
</script>
```

[Check Demo 201.html](#)

Promise chaining with Async/await in ES8

```
<script>
function add_positivenos_async(n1, n2) {
  let p = new Promise(function (resolve, reject) {
    if (n1 >= 0 && n2 >= 0) {
      //do some complex time consuming work
      resolve(n1 + n2)
    } else
      reject('NOT_Postive_Number_Passed')
  })
  return p;
}

async function addInSequence() {
  let r1 = await add_positivenos_async(10, 20)
  console.log("first result", r1);
  let r2 = await add_positivenos_async(r1, r1);
  console.log("second result", r2)
  let r3 = await add_positivenos_async(r2, r2);
  console.log("third result", r3)
  return "Done Sequence"
}

addInSequence().then((r)=>console.log("Async :",r));
console.log('end')
</script>
```

The output of the above code

```
end
first result 30
second result 60
third result 120
Async : Done Sequence
```

ES9

- ▶ **ES9 introduced:**
 - ▶ Rest/Spread Properties
 - ▶ Promise: finally()

ES9 - Rest/Spread Properties

```
<script>
  const student = {
    age:10,
    height:5,
    weight:50
  }
  const {age,...other} = student;
  console.log(age)
  console.log(other)
</script>
```

The output of the above code will be as given below –

```
10
{height: 5, weight: 50}
```

The value of age property of student is copied into the age variable while the values of the remaining properties are copied into the other variable using the rest syntax `...`

ES9 – Promise Finally

The **finally()** is executed whenever a promise is settled, regardless of its outcome. This function returns a promise. It can be used to avoid code duplication in both the promise's **then()** and **catch()** handlers.

Syntax

The below mentioned syntax is for the function **finally()**.

```
promise.finally(function() {  
  });  
promise.finally(()=> {  
  });
```


ES9 – promise finally() Example

```
<script>
  let asyncSquareFn = function(n1){
    return new Promise((resolve,reject)=>{
      setTimeout(()=>{
        if(n1>=0){
          resolve(n1*n1)
        }
        else reject('NOT_POSITIVE_NO')
      },3000)
    })
  }
  console.log('Start')

  asyncSquareFn(10)//modify to add -10
  .then(result=>{
    console.log("result is",result)
  }).catch(error=>console.log(error))
  .finally(() =>{
    console.log("inside finally")
    console.log("executes all the time")
  })

  console.log("End");
</script>
```

The following example declares a async function that returns the square of a positive number after a delay of 3 seconds. The function throws an error if a negative number is passed. The statements in the finally block is executed in either case, whether the promise is rejected or resolved.

The output of the above code

```
Start
End
//after 3 seconds
result is 100
inside finally
executes all the time
```

ES10 – ECMAScript2019

- ▶ ES10 introduced some new functions/features:
 - ▶ `flat()` is a method which is used to flatten an array. There are certain situations in which the elements of an array are an array. These types of arrays are called nested arrays.
 - ▶ Normally to un-nest (flatten) them, we had to use recursion. Now with the introduction of `flat()`, it can be done in a single line. **F.Y.I – a flattened array is an array of depth 0**

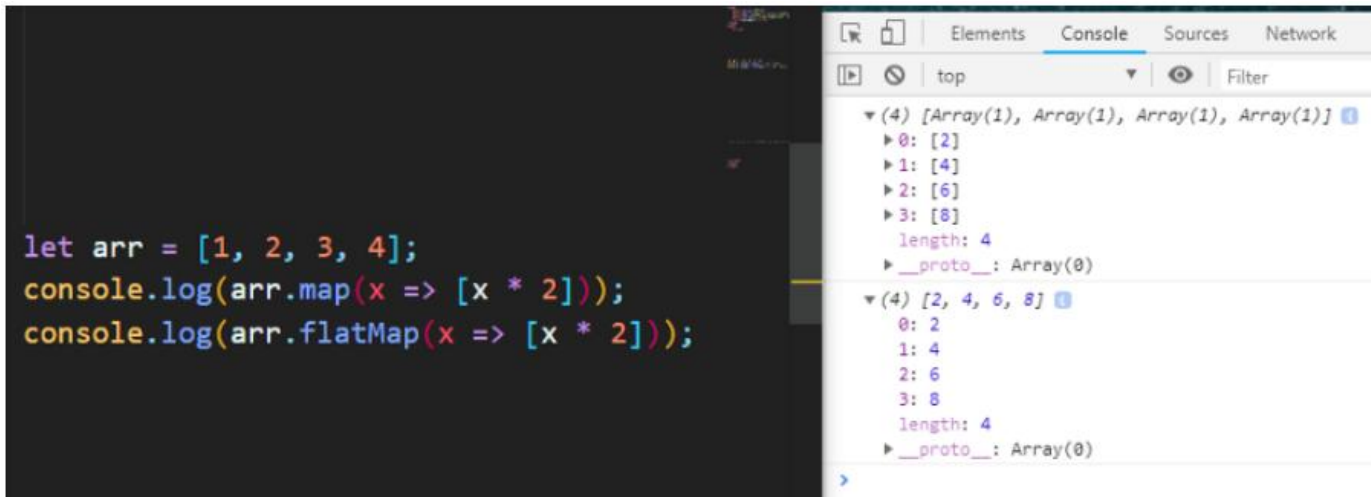


```
var arr=[1,2,3,[4,5,6,[7,8,9,[1,2,3]]]];
console.log(arr.flat(3))
```

The image shows a code editor with a dark background. On the left, there is a JavaScript code snippet: `var arr=[1,2,3,[4,5,6,[7,8,9,[1,2,3]]]]; console.log(arr.flat(3))`. On the right, there is a console output showing the result of the `flat(3)` method. It displays an array of 12 elements: `[1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 2, 3]`. Below the array, it shows the indices from 0 to 11 and the `length` property set to 12.

ES10 – ECMAScript2019

- ▶ ES10 introduced some new functions/features:
 - ▶ **flatMap()** is used to flatten a nested array and to change the values according to a function like a map() function.



ES11 – ECMAScript2020

- ▶ ES11 introduced private class variables:

```
class HelloWorld {  
  #message = "How are you?"; //this is a private variable  
  getMessage()  
  {  
    console.log(this.#message)  
  }  
}
```

```
const hello = new HelloWorld()  
hello.getMessage() // How are you?
```

ES12, ES13 and ES14

- ▶ Small but useful features have been added to ES12, ES13 and now also in 2023 ES14
- ▶ You can check them online.



AJAX

AJAX

- ▶ AJAX is not a programming language.
- ▶ AJAX is a technique for accessing web servers from a web page.
- ▶ AJAX stands for Asynchronous JavaScript And XML.

AJAX Example

HTML Page

```
▶ <!DOCTYPE html>
  <html>
  <body>

  <div id="demo">
    <h2>Let AJAX change this text</h2>
    <button type="button" onclick="loadDoc()"
">Change Content</button>
  </div>

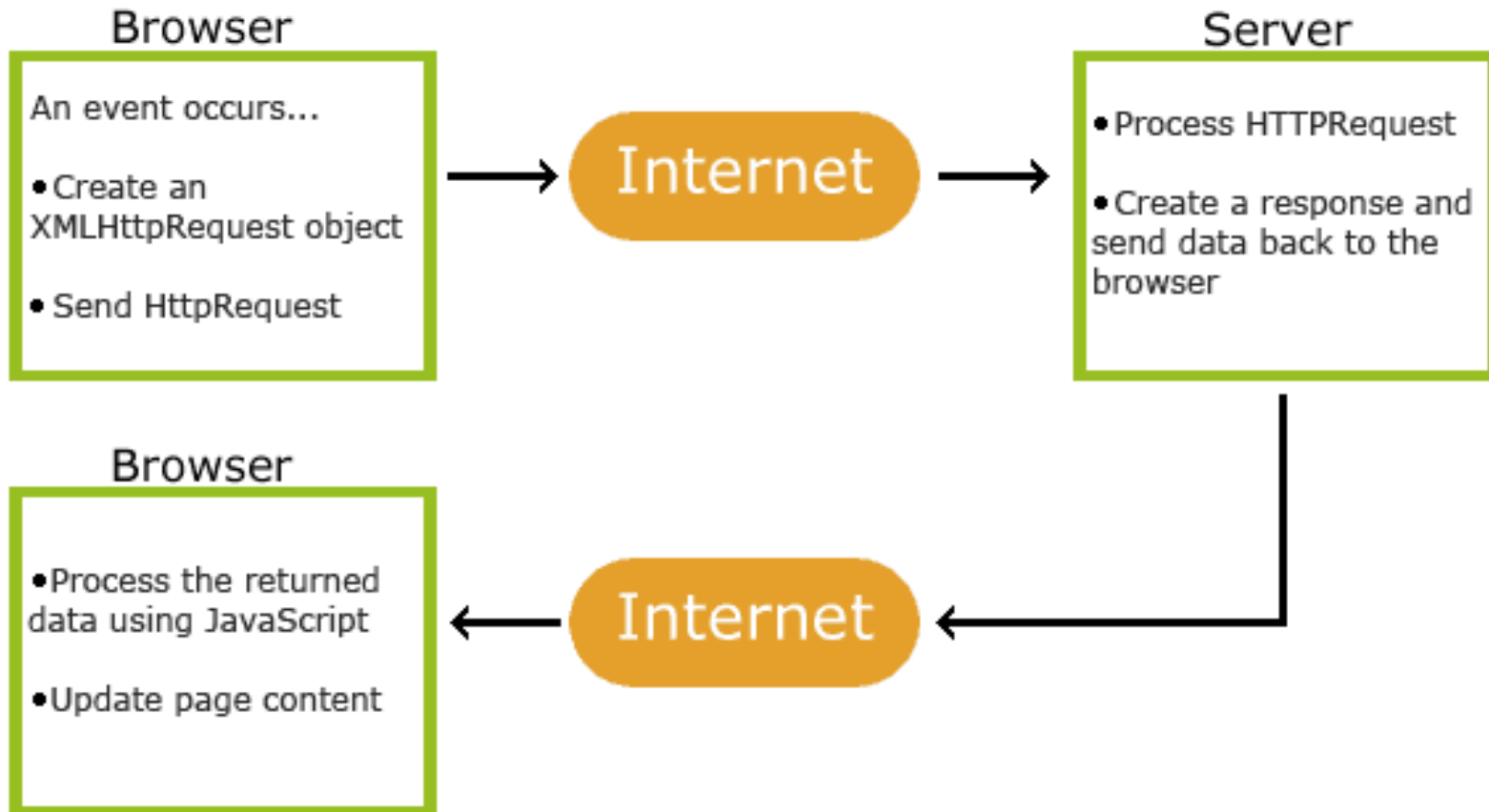
  </body>
</html>
```


AJAX Example

- ▶ Function loadDoc()
- ▶

```
function loadDoc() {  
    var xhttp = new XMLHttpRequest();  
    xhttp.onreadystatechange = function() {  
        if (this.readyState == 4 && this.status ==  
200) {  
            document.getElementById("demo").innerHTML  
= this.responseText;  
        }  
    };  
    xhttp.open("GET", "ajax_info.txt", true);  
    xhttp.send();  
}
```

What is AJAX?



AJAX Cycle

1. An event occurs in a web page (the page is loaded, a button is clicked)
2. An XMLHttpRequest object is created by JavaScript
3. The XMLHttpRequest object sends a request to a web server
4. The server processes the request
5. The server sends a response back to the web page
6. The response is read by JavaScript
7. Proper action (like page update) is performed by JavaScript

Next

- ▶ MVC
- ▶ Python (Back-end Programming Language)
- ▶ Django (Web Applications Framework)